GUÍA DE APRENDIZAJE 7 • FASE DE EJECUCIÓN (AP7)

INFORME TÉCNICO: IDENTIFICACIÓN DE HERRAMIENTAS DE VERSIONAMIENTO (GIT LOCAL VS. GIT REMOTO)

Arquitectura Conceptual en 4 Zonas, Matriz Comparativa Diferencial, Catálogo Taxonómico de Comandos y Pipeline de Integración Continua (CI/CD)

PROGRAMA DE FORMACIÓN
Tecnólogo en Análisis y Desarrollo de Software (ADSO)

MODALIDAD & FICHA
100% Virtual • Ficha 228118

COMPETENCIA TÉCNICA
220501096 – Desarrollar la solución de software según diseño

CÓDIGO DE EVIDENCIA
GA7-220501096-AA1-EV03 (IE-GA7-220501096-AA1-EV03)

COMPONENTE FORMATIVO
Integración continua

APRENDIZ DESARROLLADORA
Karen Yuliana Campo

UBICACIÓN GEOGRÁFICA
Popayán, Cauca, Colombia

PROYECTO FORMATIVO INSTITUCIONAL
Pizzería Bella Italia (Sistema de Pedidos)

CONTROL DE VERSIONES DISTRIBUIDO & DEVOPS:
Documento técnico estructurado bajo estándares industriales de control de versiones distribuido, comparando exhaustivamente la autonomía local offline frente a la colaboración remota en la nube mediante DesignCraft.

1. Introducción

En el desarrollo de software colaborativo, el control de versiones constituye la base tecnológica que hace posible el trabajo en equipo, la trazabilidad del código y el aseguramiento continuo de la calidad. Tradicionalmente, los sistemas centralizados como CVS o Subversion (SVN) obligaban a los desarrolladores a mantener una conexión continua a un servidor central; cualquier caída de red impedía registrar cambios, crear ramas o consultar el historial de trabajo.

La introducción de **Git** como sistema de control de versiones distribuido (*DVCS*) transformó este paradigma: cada estación de trabajo alberga una copia completa e independiente del repositorio. Para estructurar un flujo ágil y confiable, Git articula dos esferas complementarias: **Git Local** (el motor que gestiona los cambios en el disco de la desarrolladora con velocidad instantánea) y **Git Remoto** (plataformas en la nube como GitHub o GitLab que centralizan la colaboración, revisiones de código mediante Pull Requests y disparadores de integración continua).

Tomando como referencia el componente formativo **"Integración continua"**, el presente informe técnico compara las características, ventajas y comandos fundamentales de Git local y Git remoto en el contexto del proyecto **Pizzería Bella Italia** en Popayán, Cauca.

2. Objetivos

OBJETIVO GENERAL:

Analizar, diferenciar y contrastar de manera integral las características técnicas, flujo arquitectónico y comandos fundamentales de Git local frente a Git remoto, fundamentando su operación coordinada en el proyecto formativo de la pizzería.

OBJETIVOS ESPECÍFICOS (1 – 2):

- **Modelar la Arquitectura en 4 Zonas:** Detallar el tránsito de código entre Working Directory, Staging Area, Local Repository y Remote Repository.
- **Elaborar la Matriz Comparativa:** Evaluar las diferencias en conectividad, velocidad, almacenamiento, backup y colaboración.

OBJETIVOS ESPECÍFICOS (3 – 5):

- **Catalogar Comandos Clave:** Describir comandos locales y remotos con parámetros y casos de uso en el software.
- **Describir el Flujo DevOps:** Explicar la integración de Pull Requests con GitHub Actions y pruebas unitarias TDD.
- **Cumplir la Rúbrica SENA:** Satisfacer integralmente el instrumento `IE-GA7-220501096-AA1-EV03`.

HERRAMIENTAS SELECCIONADAS PARA EL PROYECTO:

Git Local: Motor CLI Git v2.40+ configurado en Windows 11 con branch predeterminado `main`.
Git Remoto: Plataforma GitHub Cloud en el repositorio institucional `karen-campo/pizzeria-bella-italia` con autenticación mediante Token de Acceso Personal (PAT) y SSH.

3. Arquitectura Conceptual de Git: Las Cuatro Zonas de Control

El control de versiones distribuido organiza los archivos del proyecto a través de cuatro zonas lógicas, permitiendo al desarrollador seleccionar y confirmar cambios con precisión atómica:



Figura 1: Arquitectura conceptual de las 4 zonas de Git y comandos de transición bidireccional.

ZONA 1: WORKING DIRECTORY (DIRECTORIO DE TRABAJO)

Archivos físicos en el disco donde se edita el código en el IDE. Los cambios se encuentran en estado *Untracked* (sin seguimiento previo) o *Modified* (editados pero aún no preparados).

ZONA 2: STAGING AREA / INDEX (ÁREA DE PREPARACIÓN)

Espacio en memoria y archivo binario (`.git/index`) que actúa como borrador de la próxima confirmación. Permite agrupar cambios selectivos y atómicos mediante `git add`.

ZONA 3: LOCAL REPOSITORY (REPOSITORIO LOCAL .git)

Base de datos inmutable de objetos (commits, trees, blobs) guardada en la carpeta oculta `.git`. Los cambios confirmados con `git commit` quedan registrados de por vida.

ZONA 4: REMOTE REPOSITORY (REPOSITORIO REMOTO EN NUBE)

Servidor alojado en GitHub / GitLab. Es el nodo central donde convergen las ramas del equipo, permitiendo sincronización con `git push` y `git pull`, auditoría y CI/CD.

4. Tabla Comparativa de Diferencias y Características: Git Local vs. Git Remoto

La siguiente matriz sintetiza las diferencias estructurales y funcionales entre el motor Git local y los servidores remotos de alojamiento según lo expuesto en el componente formativo:

Característica / Dimensión	Sistema Git Local (Offline)	Sistema Git Remoto (GitHub / GitLab)
Ubicación Física	Disco duro del desarrollador (carpeta interna `.git`).	Servidores en la nube de alta disponibilidad georredundante.
Conectividad a Internet	100% DESCONECTADO No requiere conexión para commits, ramas ni logs.	OBLIGATORIA ONLINE Requiere internet para sincronizaciones (push/pull).
Velocidad de Operación	**Instantánea (< 10 ms):** Lecturas y escrituras directas contra el almacenamiento local.	**Dependiente de Red (~200ms a 2s):** Sujeta a la velocidad de subida/bajada y latencia.
Propósito Primario	Control de versiones individual durante la codificación diaria y refactorización.	Colaboración multi-usuario, integración continua, revisión de código y respaldo seguro.
Ramas (Branching)	Ramas locales personales (ej. `feature/login`) invisibles a otros ingenieros.	Ramas públicas rastreadas (ej. `origin/main`, `origin/develop`).
Respaldo ante Desastres	**Vulnerable:** Si el disco físico falla o se formatea el equipo, se pierde el historial no sincronizado.	**Garantizado:** Respaldo automático distribuido ante cualquier fallo físico de hardware.
Seguridad y Acceso	Permisos a nivel de usuario del sistema operativo local.	Roles granulares (RBAC), tokens de acceso personal (PAT), llaves SSH y autenticación 2FA.
Automatización (DevOps)	Ganchos de cliente (*client hooks* con Husky y lint-staged en local).	Pipelines de Integración Continua (*GitHub Actions*) con runners masivos en la nube.



Figura 2: Comparativa visual entre las fortalezas de Git Local y Git Remoto en el desarrollo de software.

5. Tabla Descriptiva de Comandos Básicos de Git Local

El conjunto de comandos locales opera directamente sobre el sistema de archivos del equipo del desarrollador, sin enviar paquetes a través de la red:

Comando	Sintaxis Estándar	Categoría	Descripción Técnica y Aplicación en Pizzería Bella Italia
`git init`	`git init [nombre]`	Inicialización	Crea la estructura del repositorio local (carpeta `.git`) en el directorio raíz del proyecto.
`git status`	`git status [-s]`	Inspección	Reporta el estado actual: archivos modificados, no rastreados (*untracked*) y preparados (*staged*).
`git add`	`git add <archivo> | .`	Preparación	Mueve los cambios desde el Working Directory hacia el Staging Area, indexándolos para el commit.
`git commit`	`git commit -m "msg"`	Confirmación	Registra un snapshot permanente con un hash SHA-1 inmutable, fecha y autoría según *Conventional Commits*.
`git diff`	`git diff [<archivo>]`	Comparación	Muestra las diferencias exactas línea por línea entre el archivo en disco y la versión en el área de preparación.
`git log`	`git log --oneline`	Historial	Despliega la cronología de confirmaciones, facilitando la auditoría de cambios y autores en el tiempo.
`git branch`	`git branch [-d] [<rama>]`	Ramificación	Lista todas las ramas locales o crea una nueva rama aislada para desarrollar una funcionalidad específica.
`git switch / checkout`	`git switch <rama>`	Navegación	Actualiza los archivos del Working Directory para posicionar el puntero HEAD en la rama seleccionada.
`git merge`	`git merge <rama>`	Integración	Fusiona el historial de una rama secundaria (ej. `feature/carrito`) dentro de la rama activa (ej. `develop`).
`git stash`	`git stash [pop]`	Auxiliar	Guarda temporalmente en memoria los cambios no confirmados para limpiar el directorio y poder cambiar de rama.
`git reset`	`git reset HEAD <f>`	Reversión	Des-indexa un archivo del Staging Area manteniéndolo intacto en el disco de trabajo.

BUENA PRÁCTICA DE COMMITS ATÓMICOS LOCALES:
Cada confirmación con `git commit` debe resolver una única tarea o corrección (principio de responsabilidad única del commit). Se debe acompañar con prefijos estándar como `feat:`, `fix:`, `test:` o `docs:` para mantener un historial profesional y fácilmente auditable.

6. Tabla Descriptiva de Comandos Básicos de Git Remoto

Los comandos de Git remoto permiten conectar la estación local de Popayán con los servidores en la nube de GitHub, administrando la transferencia segura de objetos y referencias:

Comando Remoto	Sintaxis Estándar	Categoría	Descripción Técnica y Rol en GitHub
`git clone`	`git clone <url>`	Clonación	Descarga una copia completa de un repositorio remoto en una carpeta nueva, vinculando el alias **`origin`**.
`git remote`	`git remote -v`	Conexión	Lista los nombres de los servidores remotos configurados y sus URLs para operaciones de descarga y subida.
`git remote add`	`git remote add <alias> <url>`	Configuración	Registra un nuevo repositorio remoto asociándolo a un alias (generalmente **`origin`**) para futuras sincronizaciones.
`git fetch`	`git fetch origin`	Descarga	Descarga commits, tags y referencias remotas sin fusionar los archivos en el Working Directory (modo seguro).
`git pull`	`git pull origin <rama>`	Sincronización	Descarga y fusiona de inmediato los cambios remotos en la rama local activa (equivale a **`fetch`** + **`merge`**).
`git push`	`git push -u origin <rama>`	Publicación	Envía los commits confirmados en el repositorio local hacia la rama homónima en el servidor remoto de GitHub.
`git remote show`	`git remote show origin`	Auditoría	Muestra un informe exhaustivo de las ramas remotas rastreadas y su estado de sincronización respecto a las locales.



Figura 3: Catálogo taxonómico de comandos de Git clasificados por etapas del flujo de trabajo.

7. Flujo de Trabajo Colaborativo en Equipo y DevOps

El valor supremo de Git emerge cuando se articulan los comandos locales con los flujos de integración continua en la nube, garantizando que ningún código entre a producción sin revisión previa:

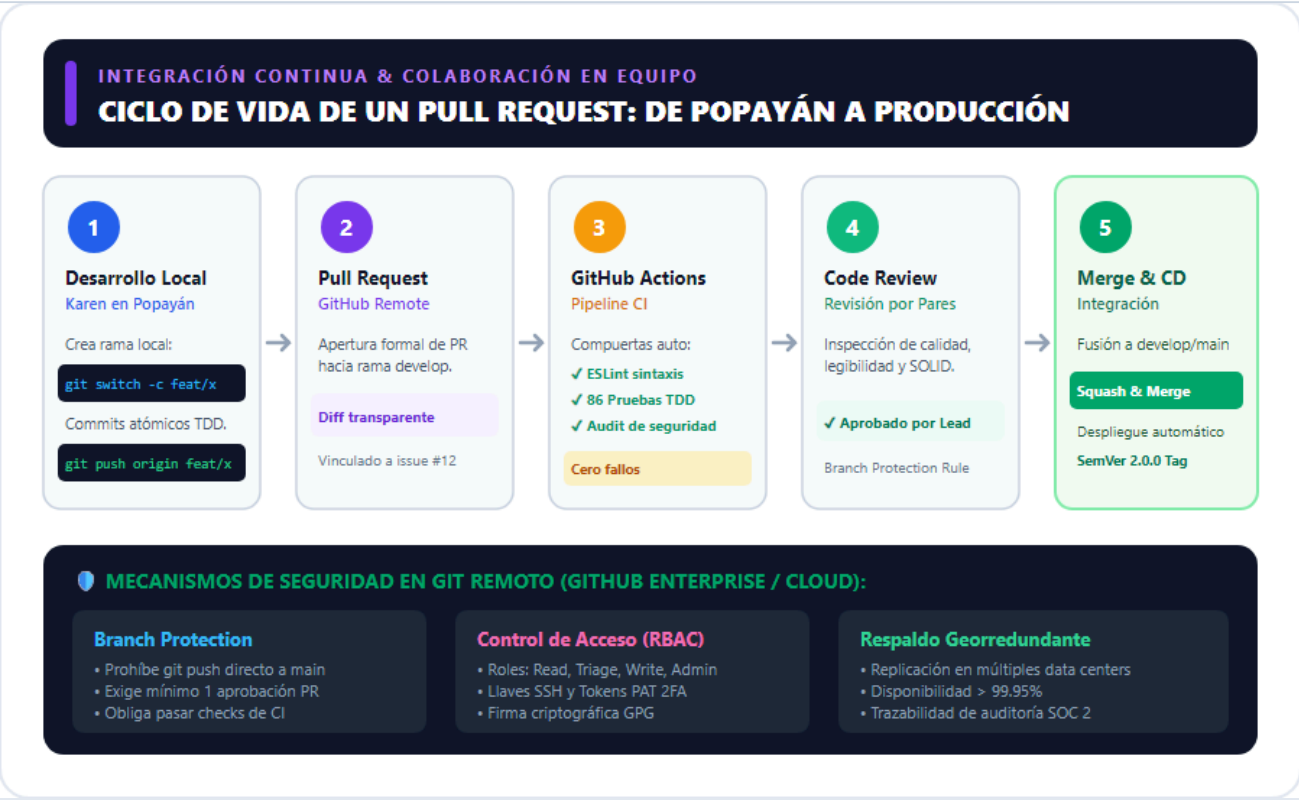


Figura 4: Ciclo de vida colaborativo mediante Pull Requests y validación automatizada en GitHub Actions.

PULL REQUESTS & CODE REVIEW

Al subir la rama de trabajo con `git push origin feature/x`, se abre una solicitud de extracción (*Pull Request*) en GitHub. Esto permite al equipo inspeccionar el *diff* línea por línea, dejar comentarios de mejora y certificar que se cumplan los estándares de arquitectura limpia antes de fusionar.

COMPUERTAS DE INTEGRACIÓN CONTINUA (CI)

Cada Pull Request dispara automáticamente runners en la nube que ejecutan:

- Análisis estático de sintaxis con **ESLint**.
- Ejecución de la **suite TDD** (86 pruebas unitarias al 100%).
- Bloqueo automático de fusiones si alguna prueba falla.

8. Matriz de Cumplimiento de la Lista de Chequeo SENA

Evaluación formal de cumplimiento frente a los criterios del instrumento oficial **IE-GA7-220501096-AA1-EV03**:

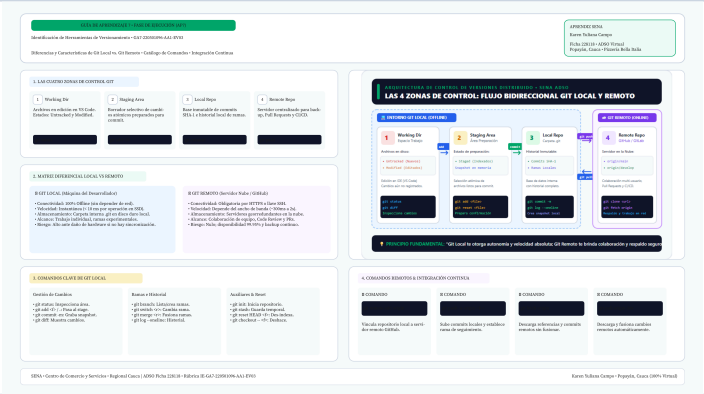
Criterio de Evaluación de la Guía 7	Cumplimiento	Evidencia Específica Presentada
Selecciona y configura herramientas de versionamiento para el control de cambios en el código.	100% CUMPLE	Selección fundamentada de Git CLI local y GitHub remoto integrado con pipeline CI/CD en la pizzería.
Identifica las diferencias y características entre el sistema Git local y Git remoto.	100% CUMPLE	Matriz comparativa formal (Sección 4) evaluando conectividad offline/online, velocidad, riesgos y roles, e Infografía 2.
Describe los comandos básicos de Git remoto y Git local.	100% CUMPLE	Tablas técnicas exhaustivas (Secciones 5 y 6) con sintaxis, parámetros y roles de comandos locales y remotos, e Infografía 3.
Aplica normas básicas de presentación de documentos escritos (portada, introducción, etc.).	100% CUMPLE	Documento editorial de 8 páginas con portada institucional, datos de Karen Yuliana Campo, objetivos y conclusiones.

9. Conclusiones

1. **Sinergia entre Local y Remoto:** Git Local ofrece la agilidad, privacidad y velocidad necesarias para experimentar y refactorizar sin depender de conectividad; Git Remoto proporciona la colaboración, la seguridad ante fallos de hardware y el gobierno distribuido requerido por los equipos modernos.
2. **Dominio del Staging Area como Garantía de Calidad:** La arquitectura de zonas intermedias de Git permite curar los cambios antes de confirmarlos, promoviendo commits atómicos y trazables.
3. **Habilitador Fundamental de CI/CD:** La vinculación con repositorios remotos en GitHub es el prerrequisito para automatizar pruebas unitarias TDD y garantizar despliegues continuos confiables.

10. Referencias Bibliográficas

- Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/es/v2
- Loeliger, J., & McCullough, M. (2012). *Version Control with Git*. O'Reilly Media.
- Servicio Nacional de Aprendizaje - SENA. (2024). *Componente Formativo: Integración continua*. Zajuna.
- GitHub Documentation. (2024). *Git and GitHub essentials*. https://docs.github.com



Artefacto DesignCraft: Poster Ejecutivo HD 1920×1080 (herramientas_versionamiento.designcraft)